面试题与参考答案（基于项目经历）

架构与系统设计

1. 你在专家培训与考试系统里如何处理开考高峰的登录压力？

答：高峰场景主要是短信登录和验证码请求。我做了两层控制： 1）Redis 滑动窗口限流，按手机号、IP、设备维度分桶，限制单位时间内的短信发送频率。 2）对短信服务做了熔断与降级，当第三方接口波动时先保护核心登录流程，避免雪崩。这样能在集中开考时保证可用性，同时控制短信成本。 效果上，峰值期开考的登录稳定率能保持在 99% 以上（基于内部监控与告警统计）。

2. WebSocket 在多实例部署下如何保证消息一致性？

答：单机 WebSocket 会话无法跨节点共享。我引入 Redis Pub/Sub 做跨节点消息总线，业务事件发布到 Redis，各节点订阅后将消息推送给本机连接的客户端。这样即便用户连接打到不同节点，也能收到实时状态更新。

3. 你是怎么设计多租户数据隔离的？

答：在 Admin.NET 中基于租户上下文注入，在数据访问层统一注入租户过滤条件。所有核心表都有租户字段，框架层通过过滤器统一拦截，避免业务代码散落过滤逻辑。对敏感操作做租户校验，确保跨租户访问不可发生。

4. 电子发票网关为什么用工厂模式？

答：第三方开票渠道接口差异大，如果用 if/else 直接分支会导致维护困难。我用工厂模式 + 接口抽象把不同厂商的实现封装起来，新增供应商只需新增实现类并注册，符合开闭原则，降低改动风险。

5. 你提到“轻量框架替代重型框架”，具体怎么评估的？

答：主要看业务迭代频率、团队人数、性能与可扩展性。重型框架带来完善的基础设施，但学习成本和定制成本高。我们评估后选择 Admin.NET 是为了快速交付中后台能力，同时保留自定义扩展点。迁移时先抽象公共能力（权限、租户、审计），再逐步迁移业务模块。

数据库与性能优化

6. 你优化报表查询从 1000ms 到 200ms 的关键手段是什么？

答：主要是三点： 1）梳理最耗时 SQL，建立复合索引并减少回表。 2）对动态筛选报表提前做数据模型拆分，避免复杂 JOIN。 3）引入 Redis 对热点报表结果短时缓存，减少重复计算。

7. 你如何判断一个 SQL 需要索引？

答：先看执行计划，关注全表扫描、排序和回表。高频查询、过滤条件固定的字段适合建索引。也会注意写入频率，避免过多索引影响写性能。最终是基于执行计划和实际慢查询数据来决策。

8. MongoDB 为什么适合做第三方接口缓存？

答：第三方接口返回的字段经常变化，MongoDB 无模式更灵活；加 TTL 索引可以自动过期，维护成本低。对于结构不固定的 JSON 响应特别合适。 落地效果是第三方 API 调用次数下降约 50%，按峰值日均调用计算节省最明显。

并发、消息与异步

9. 你们的视频转码如何做到不影响主业务?

答：上传完成后只写入转码任务，主流程异步完成。转码由后台队列消费，FFmpeg 转码完成后用 Webhook 回调更新状态。这样主业务不阻塞，系统峰值压力被分摊。 在改造后，转码任务平均等待时长下降约 40%，高峰期排队明显减少。

10. Redis 限流为什么用滑动窗口而不是固定窗口?

答：固定窗口容易在边界出现突刺，例如 59 秒和 60 秒各发一批。滑动窗口更平滑，能更真实反映单位时间内的请求密度，控制更精确。

11. 使用 MQ 时你关注哪些可靠性问题?

答：主要是消息丢失、重复、顺序和消费失败。会确保消息持久化、消费确认机制、幂等处理、失败重试或死信队列，避免业务异常导致数据不一致。

安全与合规

12. 你在金融保函系统里做了哪些安全措施?

答：对外报文使用 SM4 对称加密保护内容，再加摘要校验防篡改。接口设计上对请求做签名验证，保证报文未被篡改，并记录审计日志便于追踪。

13. 你们系统如何处理敏感信息存储?

答：敏感字段采用脱敏展示，存储层可做加密或哈希处理。访问敏感数据前严格鉴权，并记录审计日志，确保合规性。

工程实践与协作

14. 你在 DevOps 方面具体做过什么?

答：搭建 Jenkins 流水线完成自动构建与发布；Docker 镜像构建后推送 Harbor；使用 Helm 编排部署到 K8s；结合 Linux 命令进行线上日志定位和性能排查。

15. 你提到“从 0 到 1”交付项目，如何控制风险?

答：会先明确业务边界和核心流程，设计 MVP 方案；拆分里程碑，优先交付核心路径；同时在框架层预留扩展点，减少后期重构成本。 以 HDIS 为例，引入 Admin.NET 后业务模块交付周期缩短约 40%，主要来自脚手架能力与通用模块复用。

16. 你怎么保证接口设计的一致性?

答：统一接口规范（状态码、错误码、分页格式），封装通用响应结构；业务层强制使用统一异常处理和日志跟踪；API 文档通过自动生成保证一致性。

前端与全栈协作

17. 你在 Vue3 中后台项目里主要负责哪些内容？

答：主要是页面模块搭建、表格与表单组件封装、接口联调、权限控制与路由动态加载。对接后端接口时会约定统一的错误码与分页规范。

18. 你如何设计前后端权限体系？

答：后端控制数据与操作权限，前端负责菜单和路由显示。RBAC 角色权限通过接口下发，前端根据权限渲染菜单和按钮，关键操作仍以后端校验为准。

业务理解与问题排查

19. 线上故障你如何排查？

答：先确认影响范围与复现条件，查看监控与日志；根据异常堆栈定位模块；必要时回溯最近发布的变更；若是性能问题，再看 CPU/内存/线程/SQL 执行计划等指标。

20. 如果 Redis 挂了怎么办？

答：核心流程不能强依赖缓存。缓存作为加速层，Redis 不可用时走数据库直查，并通过降级保护。对于限流这种强依赖场景，需要预置保护策略，比如限流阈值下发和降级策略。

设计模式与代码质量

21. 你用过哪些设计模式，实际场景是什么？

答：工厂模式用于第三方渠道对接的可扩展设计；策略模式用于根据业务条件选择不同处理逻辑；在网关层使用适配器模式统一多来源接口。

22. 你如何保证系统可维护性？

答：模块边界清晰、接口稳定、公共能力抽象；通过统一日志、错误处理、代码规范和自动化测试提升质量，同时避免业务逻辑与基础设施耦合。

综合类

23. 为什么你能胜任 Java 技术栈迁移？

答：底层能力是语言无关的，比如并发模型、设计模式、数据库优化、架构设计。对 Java 的学习重点会放在生态差异与常用框架的实践上，迁移成本主要是语法与生态习惯的切换。

24. 你怎么看待“技术广但不深”的质疑？

答：我把核心深度放在 .NET 后端和系统架构上，其他技术是为了交付完整性和协作效率。广度不是分散，而是为了在关键场景能快速落地和排障。

25. 未来想重点提升哪些方向？

答：持续深化微服务治理能力、分布式一致性、云原生监控告警体系，并在高并发场景下积累更多稳定性建设经验。